HumanEval

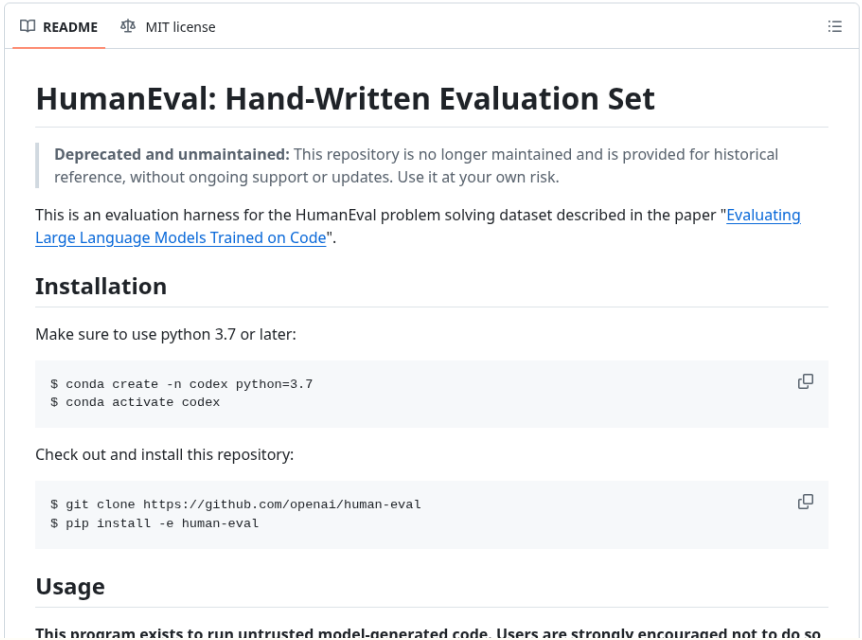
Hand-Written Evaluation Set

仕様を読み、実行テストを通る

Python 関数を生成する

sasakiy84

HumanEval の公式リポジトリ



README MIT license

HumanEval: Hand-Written Evaluation Set

> **Deprecated and unmaintained:** This repository is no longer maintained and is provided for historical reference, without ongoing support or updates. Use it at your own risk.

This is an evaluation harness for the HumanEval problem solving dataset described in the paper "Evaluating Large Language Models Trained on Code".

Installation

Make sure to use python 3.7 or later:

```
$ conda create -n codex python=3.7
$ conda activate codex
```

Check out and install this repository:

```
$ git clone https://github.com/openai/human-eval
$ pip install -e human-eval
```

Usage

This program exists to run untrusted model-generated code. Users are strongly encouraged not to do so

[1] OpenAI. HumanEval: Hand-Written Evaluation Set. 改訂 b9453e8 、 2026-10-17 参照。 https://github.com/openai/human-eval

1. 関数を完成させる課題と実例
2. 実行テストと pass@k の読み方
3. 問題の作成とデータの構成
4. テストの拡充と現在の利用

1. 関数を完成させる課題と実例

関数の仕様を読み、処理の本体を生成する

- HumanEval は、人が作成した 164 問の Python 関数の完成課題である。 OpenAI が 2021 年に公開した。
- 関数名、引数、型注釈、英語の仕様文、入出力例などが入力となる。モデルはその続きを生成して関数を完成させる。
- 生成した関数を実行し、各問に付属するテストを全て通るかで判定する。

[1] Chen et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374. https://arxiv.org/abs/2107.03374

入力・参照実装・テストを別々に持つ

フィールド	役割
prompt	モデルに渡す関数の冒頭と仕様
canonical_solution	配布された参照実装
test	正しさを判定するテストコード
entry_point	評価する関数の名前

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

実例 1 ：数値の近さを判定する

仕様文の原文（ docstring から抜粋）

> Check if in given list of numbers, are any two numbers closer to each other than given threshold.

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/0 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

実例１：日本語訳と入出力例

数値のリストに、差の絶対値がしきい値より小さい 2 要素があれば True を返す。しきい値と等しい差は条件を満たさない。

```
has_close_elements([1.0, 2.0, 3.0], 0.5)
# False
has_close_elements(
    [1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)
# True
```

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/0 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

実例２：括弧のまとまりを分ける

仕様文の原文（ docstring から抜粋）

> Input to this function is a string containing multiple groups of nested parentheses. Your goal is to separate those group into separate strings and return the list of those. Separate groups are balanced (each open brace is properly closed) and not nested within each other Ignore any spaces in the input string.

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/1 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

実例２：日本語訳と入出力例

空白を無視して括弧列を読み、各グループを文字列に分け、そのリストを返す。各グループの括弧は対応して閉じており、グループ同士は入れ子にならない。

```
separate_paren_groups('( ) (( )) (( )( ))')
# ['()', '((()))', '((()()))']
```

開き括弧で深さを増やし、閉じ括弧で減らす。深さが 0 に戻るたびに 1 グループを確定する。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/1 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

実例３：途中の残高を調べる

仕様文の原文（ docstring から抜粋）

> You're given a list of deposit and withdrawal operations on a bank account that starts with zero balance. Your task is to detect if at any point the balance of account fallls below zero, and at that point function should return True. Otherwise it should return False.

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/3 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

実例３：日本語訳と入出力例

残高 0 から入出金のリストを順に適用する。途中で一度でも残高が負になれば True 、ならなければ False を返す。

```
below_zero([1, 2, 3])
# False
below_zero([1, 2, -4, 5])
# True
```

最終残高だけでなく、更新のたびに負になったかを判定する。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/3 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

しきい値「未満」と「以下」では答えが変わる

2 要素	差の絶対値	しきい値	返り値
[1.0, 1.5]	0.5	0.6	True
[1.0, 1.5]	0.5	0.5	False

HumanEval/0 の仕様に基づく境界の例。差がしきい値と等しいときは False 。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

最終残高が正でも、途中で負なら True となる

適用した入出金	その時点の残高
開始前	0
+1	1
+2	3
−4	−1
+5	4

HumanEval/3 の入力 [1, 2, −4, 5] 。残高が− 1 になった時点で True を返す。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

2. 実行テストと pass@k の読み方

1 候補の正しさは、全テストの通過で判定する

- 生成した関数を、問題に付属するテストコードへ渡す。全ての検査を通れば、その候補は成功となる。
- 参照実装との文字列の一致は求めない。同じ入力に対して求められた出力を返す別の実装も正解になる。
- 構文エラー、実行時エラー、時間切れ、期待値との不一致があれば、その候補は失敗となる。

[1] OpenAI. HumanEval: evaluation.py. 改訂 b9453e8 。 https://github.com/openai/human-eval/blob/b9453e8/human_eval/evaluation.py

pass@k は、 k 候補に正解が含まれる確率を表す

- 同じ問題に対して k 個の候補を生成し、少なくとも 1 個が全テストを通る確率を考える。
- 各問で n 個の候補を生成し、そのうち c 個が成功したとき、重複なく k 個を選んで全て失敗する確率を用いて推定する。
- 問題ごとの推定値を平均して、 HumanEval 全体の pass@k として報告する。 k や生成条件をそろえて値を比較する。

[1] Chen et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374. https://arxiv.org/abs/2107.03374

10 候補のうち 2 候補が成功した場合

pass@k = 1 − C(n−c, k) / C(n, k)

C(a, b) は a 個から b 個を選ぶ組合せの数。 n = 10 、 c = 2 として計算する。

k	計算	推定値
1	1 − 8 / 10	20.0%
5	1 − 56 / 252	77.8%

[1] Chen et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374. n は k 以上とする。 k=5 は成功候補を選び出す能力の指標ではない。
https://arxiv.org/abs/2107.03374

2021 年： Codex-12B は候補数で成績が変わる

指標	成績
pass@1	28.81%
pass@10	46.81%
pass@100	72.31%

原著 Table 1 。各指標には論文が選んだ生成温度を使う。 12B は約 120 億パラメータ。

[1] Chen et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374. https://arxiv.org/abs/2107.03374

2024 年：元のテストと強化版で差が出る

モデル	HumanEval	HumanEval+
Qwen2.5-Coder-32B-Instruct	92.7%	87.2%
GPT-4o-2024-08-06	92.1%	86.0%
Claude-3.5-Sonnet-20241022	92.1%	86.0%

Qwen2.5-Coder 報告 Table 16 の pass@1 。 HumanEval+ はテストを拡充した同じ 164 問。

[1] Hui et al. (2024). Qwen2.5-Coder Technical Report. arXiv:2409.12186. https://arxiv.org/abs/2409.12186

高い得点を読むときは、テストの範囲も見る

- HumanEval は、短い Python 関数が指定された入力に対して正しく振る舞うかを評価する。テストを通る候補が成功となる。
- HumanEval+ はテストの入力を増やし、元のテストで見逃した誤りを検出する。
- 未知の仕様や大きなコードベースを扱う能力は、公開問題の得点だけでは分からない。

[1] Chen et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374. https://arxiv.org/abs/2107.03374

[2] Liu et al. (2023). Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. NeurIPS. https://arxiv.org/abs/2305.01210

3. 問題の作成とデータの構成

公開コードの記憶だけに頼らない評価を目指した

- 原著は、 GitHub 由来のコードで学習した Codex を評価するため、問題を手作業で新しく作成した。
- 仕様の読解、推論、アルゴリズム、簡単な数学を必要とする問題を含む。関数の署名と docstring 、参照実装、テストをそろえた。
- 原著は 1 問あたり平均 7.7 件のテストを報告する。モデルが仕様を満たすかを、実際の実行結果で調べる。

[1] Chen et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374. https://arxiv.org/abs/2107.03374

公式の問題ファイルは評価用の 164 問

項目	配布内容
問題数	164 問
識別子	HumanEval/0 〜 HumanEval/163
対象言語	Python
公式の学習用分割	用意されていない

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

prompt には補助関数が含まれる問題もある

- prompt は、関数名や英語の仕様文だけでなく、import や型注釈、入出力例、既存の補助関数を含むことがある。
- HumanEval/10 では、回文を判定する補助関数 is_palindrome が入力に含まれる。生成する make_palindrome から利用できる。
- 生成する本体と、もともと入力に含まれるコードを区別すると、モデルが何を完成させるかを把握しやすい。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

入力の長さには問題間の幅がある

対象 164 問	中央値 25 〜 75% 点	最大
prompt の文字数	396 288 〜 567	1,360
prompt の行数	13 10 〜 15	36

文字数は空白・改行を含む。行数は splitlines() で計測。

[1] OpenAI. HumanEval.jsonl.gz を 164 問全件集計。改訂 b9453e8 。 https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

4. テストの拡充と現在の利用

HumanEval+ は同じ仕様のテストを増やす

- EvalPlus （2023）は、元の入力を変形する方法とモデルによる入力生成を組み合わせ、 HumanEval のテストを約 80 倍に増やした。
- 人が入力の妥当性や参照実装を点検し、必要な訂正も加えた。新しい関数の仕様を 164 問追加するものではない。
- 元のテストで通ったコードの一部が強化版では失敗し、評価値やモデルの順位が変わることを示した。

[1] Liu et al. (2023). Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. NeurIPS. https://arxiv.org/abs/2305.01210

短い関数の評価から、実際の変更課題へ広がる

- HumanEval は、 1 関数の生成結果を、仕様と実行テストで評価する代表的な基準として使われてきた。
- SWE-bench は、実際の GitHub リポジトリの課題を解決する変更を評価する。複数のファイルや既存実装の理解が必要になる。
- 2026-10-17 時点でも EvalPlus は HumanEval と HumanEval+ の評価手順を公開している。元の OpenAI リポジトリは保守終了を明記している。

[1] EvalPlus. EvalPlus: Rigorous Evaluation of LLM-synthesized Code. 改訂 26d6d00 、 2026-10-17 参照。 https://github.com/evalplus/evalplus
[2] Jimenez et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR. https://arxiv.org/abs/2310.06770

公式の問題を読み、入力と参照実装を見る

```
from human_eval.data import read_problems

problems = read_problems()
p = problems["HumanEval/0"]
print(p["prompt"])
print(p["canonical_solution"])
```

公式パッケージの読み込み例。配布フィールドを確認するためのコード。

[1] OpenAI. HumanEval: Hand-Written Evaluation Set. 改訂 b9453e8 、 2026-10-17 参照。 https://github.com/openai/human-eval

配布物は MIT License で利用できる

- 公式リポジトリでは、問題ファイルと読み込み・評価コードを MIT License で公開している。
- 候補は、 task_id と completion を持つ JSON Lines として保存する。 completion には prompt に続く関数本体を入れる。
- 公式 README に従い、生成コードは隔離した環境で実行する。

[1] OpenAI. HumanEval: Hand-Written Evaluation Set. 改訂 b9453e8 、 2026-10-17 参照。 https://github.com/openai/human-eval

仕様の細部を、実行できる振る舞いに変える

- HumanEval は、英語の仕様や入出力例から Python 関数を完成させる、 164 問の評価データである。
- しきい値との等号、途中の状態、空入力など、条件を満たす実装かをテストで判定する。
- 成績は pass@k の候補数と生成条件をそろえて読み、元のテストか、 HumanEval+ の強化テストかも確認する。

[1] Chen et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374. https://arxiv.org/abs/2107.03374

Appendix

1. 仕様の異なる追加 10 例
2. 入力と参照実装の集計
3. 研究と参考文献

1. 仕様の異なる追加 10 例

追加例 1 ：小数部分を取り出す

仕様文の原文（ docstring から抜粋）

> Given a positive floating point number, it can be decomposed into and integer part (largest integer smaller than given number) and decimals (leftover part always smaller than 1). Return the decimal part of the number.

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/2 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 1 ：日本語訳と入出力例

正の浮動小数点数を受け取り、整数部分を除いた、 1 未満の小数部分を返す。

```
truncate_number(3.5)
# 0.5
```

配布された参照実装は、 1.0 で割った余りを返す。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/2 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 2 ：平均絶対偏差を求める

仕様文の原文（ docstring から抜粋）

> For a given list of input numbers, calculate Mean Absolute Deviation around the mean of this dataset. Mean Absolute Deviation is the average absolute difference between each element and a centerpoint (mean in this case): MAD = average | x - x_mean |

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/4 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 2 ：日本語訳と入出力例

数値のリストについて、平均からの平均絶対偏差を求める。各要素と平均との差の絶対値を取り、その平均を返す。

```
mean_absolute_deviation([1.0, 2.0, 3.0, 4.0])
# 1.0
```

平均との差の符号を打ち消すため、各差の絶対値を取ってから合計する。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/4 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 3 ：要素の間に値を挟む

仕様文の原文（ docstring から抜粋）

> Insert a number 'delimeter' between every two consecutive elements of input list `numbers'

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/5 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例３：日本語訳と入出力例

整数リストの隣り合う要素の間に、指定された整数を挟んで返す。

```
intersperse([1, 2, 3], 4)
# [1, 4, 2, 4, 3]
intersperse([], 4)
# []
```

区切り値を先頭や末尾には加えない。空リストには空リストを返す。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/5 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 4 ：括弧の最大深さを求める

仕様文の原文（ docstring から抜粋）

> Input to this function is a string represented multiple groups for nested parentheses separated by spaces. For each of the group, output the deepest level of nesting of parentheses. E.g. (()()) has maximum two levels of nesting while ((())) has three.

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/6 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 4 ：日本語訳と入出力例

空白で区切られた括弧の各グループについて、入れ子の最大深さを返す。たとえば (()()) は深さ 2 、 (((()))) は深さ 3 となる。

```
parse_nested_parens(
    '(()()) ((())) () ((())()())')
# [2, 3, 1, 3]
```

現在の深さと、そのグループで到達した最大値を別々に保持する。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/6 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 5 ：部分文字列で絞り込む

仕様文の原文（ docstring から抜粋）

> Filter an input list of strings only for ones that contain given substring

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/7 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 5 ：日本語訳と入出力例

文字列リストから、指定された部分文字列を含む要素だけを元の順に返す。

```
filter_by_substring(['abc', 'bacd', 'cde', 'array'], 'a')
# ['abc', 'bacd', 'array']
```

部分文字列が含まれるかを要素ごとに判定する。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/7 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 6 ：和と積を同時に返す

仕様文の原文（ docstring から抜粋）

> For a given list of integers, return a tuple consisting of a sum and a product of all the integers in a list. Empty sum should be equal to 0 and empty product should be equal to 1.

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/8 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 6 ：日本語訳と入出力例

整数リストの全要素の和と積をタプルで返す。空リストの和は 0 、積は 1 とする。

```
sum_product([1, 2, 3, 4])
# (10, 24)
sum_product([])
# (0, 1)
```

和を 0 、積を 1 で初期化すると、空リストも同じ処理で扱える。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/8 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 7 ：それまでの最大値を返す

仕様文の原文（ docstring から抜粋）

> From a given list of integers, generate a list of rolling maximum element found until given moment in the sequence.

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/9 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 7 ：日本語訳と入出力例

整数列を先頭から読み、各位置までに現れた値の最大値をリストにして返す。

```
rolling_max([1, 2, 3, 2, 3, 4, 2])
# [1, 2, 3, 3, 3, 4, 4]
```

直前までの最大値と今回の値を比較して更新する。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/9 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 8 ：最短の回文を作る

仕様文の原文（ docstring から抜粋）

> Find the shortest palindrome that begins with a supplied string. Algorithm idea is simple: - Find the longest postfix of supplied string that is a palindrome. - Append to the end of the string reverse of a string prefix that comes before the palindromic suffix.

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/10 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 8 ：日本語訳と入出力例

与えられた文字列で始まる最短の回文を返す。末尾側の最長の回文を探し、その前に残った部分を逆順にして末尾へ付ける。

```
make_palindrome('cat')
# 'catac'
make_palindrome('cata')
# 'catac'
```

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/10 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 9 ：ビット列の排他的論理和

仕様文の原文（ docstring から抜粋）

> Input are two strings a and b consisting only of 1s and 0s.
> Perform binary XOR on these inputs and return result also as a string.

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/11 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 9 ：日本語訳と入出力例

0 と 1 からなる二つの文字列について、各位置の排他的論理和を文字列で返す。

```
string_xor('010', '110')
# '100'
```

同じ文字同士なら 0 、異なる文字同士なら 1 を返す。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/11 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 10 ：最長の文字列を選ぶ

仕様文の原文（ docstring から抜粋）

> Out of list of strings, return the longest one. Return the first one in case of multiple strings of the same length. Return None in case the input list is empty.

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/12 。入出力例は日本語訳とともに掲載。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

追加例 10 ：日本語訳と入出力例

文字列リストで最も長い要素を返す。同じ長さなら先に現れたものを選び、空リストなら None を返す。

```
longest(['a', 'bb', 'cc'])
# 'bb'
longest([])
# None
```

最大長を求め、その長さを持つ最初の文字列を返す。

[1] OpenAI. HumanEval.jsonl.gz. 改訂 b9453e8 。 HumanEval/12 。 # 以降は期待する返り値。
https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

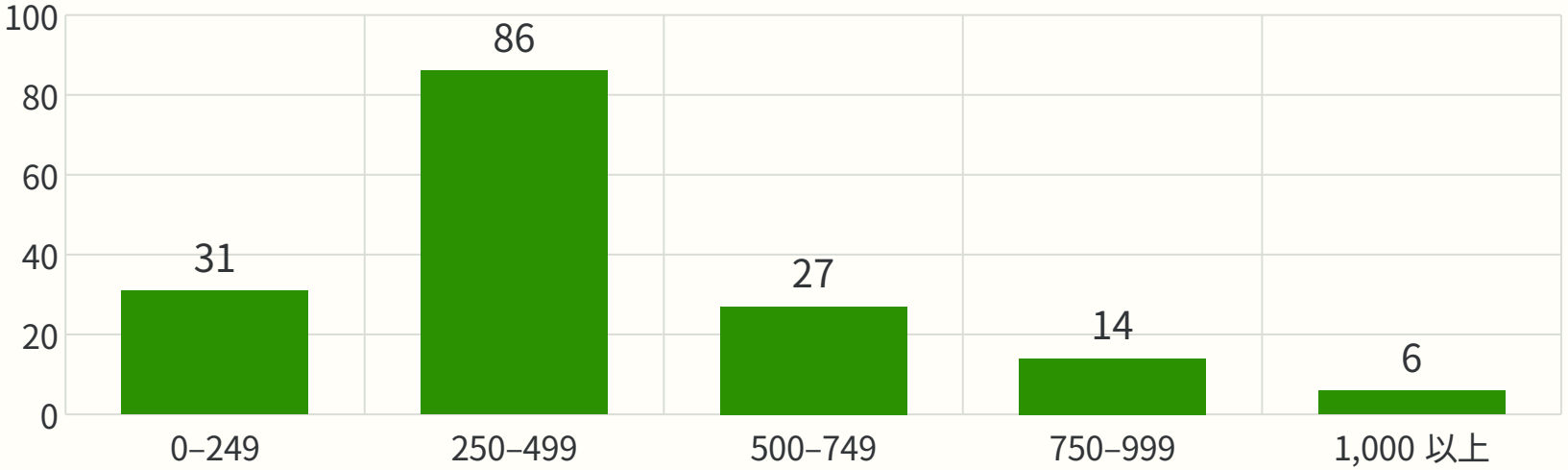
2. 入力と参照実装の集計

164 問の prompt の文字数

指標	文字数
平均	450.6
中央値	396
母標準偏差	229.6
25 〜 75% 点	288 〜 567
95% 点	926

[1] OpenAI. HumanEval.jsonl.gz を 164 問全件集計。改訂 b9453e8 。 https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

prompt は 250 〜 499 文字の問題が最も多い



横軸は prompt の文字数、縦軸は問題数。 164 問を全件集計。

[1] OpenAI. HumanEval.jsonl.gz を 164 問全件集計。改訂 b9453e8 。 https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

参照実装の長さも問題によって異なる

指標	canonical_solution の行数
平均	6.8
中央値	6
25 〜 75% 点	2 〜 10
最大	31

空行を含む splitlines() の行数。入力に含まれる署名・補助関数は含めない。

[1] OpenAI. HumanEval.jsonl.gz を 164 問全件集計。改訂 b9453e8 。 https://github.com/openai/human-eval/blob/b9453e8/data/HumanEval.jsonl.gz

3. 研究と参考文献

原著はコード生成モデルと評価方法を提示した

- Mark Chen 、 Jerry Tworek らの OpenAI の研究チームが、 Codex と、その評価用の HumanEval を発表した。
- 原著は、手書きの問題、実行による正しさの判定、複数候補から成功率を推定する pass@k を組み合わせた。
- 後続の EvalPlus はテストを強化し、同じ仕様に対する、より幅広い入力での正しさを調べる。

[1] Chen et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374. https://arxiv.org/abs/2107.03374

参考文献

Evaluating Large Language Models Trained on Code

Chen et al. (2021). OpenAI. arXiv:2107.03374.

HumanEval 、 Codex 、実行テスト、 pass@k の定義と実験を示した原著。

https://arxiv.org/abs/2107.03374

Qwen2.5-Coder Technical Report

Hui et al. (2024). arXiv:2409.12186.

2024 年のコード生成モデルを、 HumanEval と HumanEval+ などで比較する。

https://arxiv.org/abs/2409.12186

参考文献

Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation

Liu, Xia, Wang & Zhang (2023). NeurIPS.

入力の拡充と参照実装の点検により、既存テストが見逃す誤りを調べる。

https://arxiv.org/abs/2305.01210

参考文献

SWE-bench: Can Language Models Resolve Real-World GitHub Issues?

Jimenez et al. (2024). ICLR.

関数の単独生成から、実際のリポジトリの課題解決へ評価対象を広げた研究。

https://arxiv.org/abs/2310.06770

EvalPlus

EvalPlus (n.d.). 改訂 26d6d00 。 2026-10-17 参照。

HumanEval ・ HumanEval+ の評価手順、データと実装を公開する。

https://github.com/evalplus/evalplus